

# Orchestration Patterns

Five built-in ways to coordinate multiple agents — and when each earns its cost

---

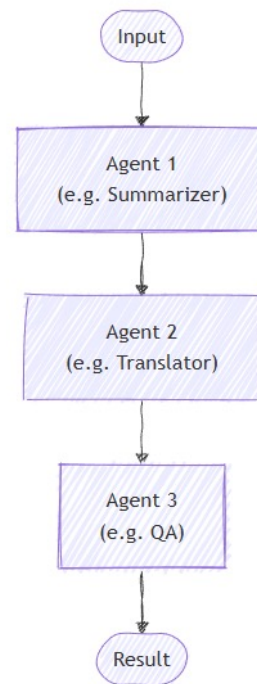
Source: <https://learn.microsoft.com/en-us/agent-framework/workflows/orchestrations/>

# Five patterns, five shapes

| Pattern    | When to use it                                                                                   |
|------------|--------------------------------------------------------------------------------------------------|
| Sequential | Agents execute one after another in a defined order — each builds on the previous agent's output |
| Concurrent | Agents execute in parallel — useful when tasks are independent and you want to reduce latency    |
| Handoff    | Agents transfer control to each other based on context — good for routing to specialists         |
| Group Chat | Agents collaborate in a shared conversation — useful for debate, review, or brainstorming        |
| Magentic   | A manager agent dynamically coordinates specialized agents — balances structure with flexibility |

# Sequential — a pipeline

- Agents run one after another; each builds on the previous agent's output
- By default, each agent consumes the full prior conversation (input + every response so far)
- `chain_only_agent_responses=True` narrows this to just the previous agent's response — useful for translation pipelines and progressive refinement
- Order Matters: agents execute strictly in the order given — there is no built-in way to route back to an earlier agent



# Sequential in code

```
from agent_framework.orchestrations import SequentialBuilder

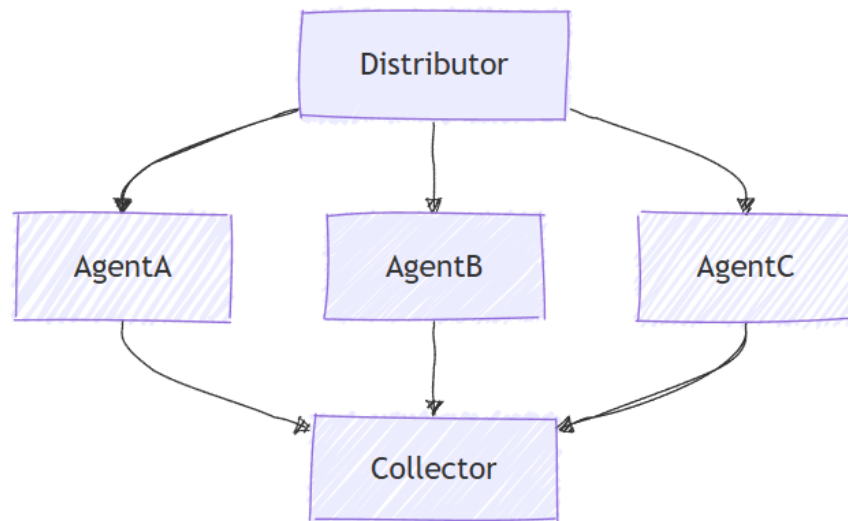
workflow = SequentialBuilder(
    participants=[drafter, editor, finalizer]).build()

result = await workflow.run("Write a brief introduction to artificial intelligence.")
```

The terminal output is the last participant's response; pass `intermediate_output_from=[...]` to also surface earlier agents' output as observational events.

# Concurrent — parallel perspectives

- Every agent processes the same input independently, at the same time
- Well suited to diverse perspectives: brainstorming, ensemble reasoning, voting systems
- The default aggregator returns one AgentResponse with one assistant message per participant — no synthesis, just collection
- Override the aggregator when you need domain-specific synthesis of the parallel results



# Concurrent in code

```
from agent_framework.orchestrations import ConcurrentBuilder

workflow = ConcurrentBuilder(participants=[researcher, marketer, legal]).build()

outputs = (await workflow.run("Launch plan for a budget electric bike.")).get_outputs()
for msg in outputs[0].messages:
    print(f"[{msg.author_name}]: {msg.text}")
```

# D E M O

~5 min

## ConcurrentBuilder proves it, wall-clock

The previous slide's code prints three agents' responses one after another, which LOOKS sequential on the page. This demo times it for real: a small AgentMiddleware timestamps each agent's start/finish under ConcurrentBuilder and draws an ASCII overlap timeline, then the SAME three agents run again with plain sequential `await agent.run(...)` calls. Concurrent's total wall-clock lands close to one agent's own time; sequential lands close to the sum of all three — that gap is the proof.

*Runbook: [demos/day4/module-2-demo-1-concurrent-parallelism.md](#)*

# Handoff — control transfers, not delegation

## Agents as tools (Module 1 recap)

- Agents as tools (Module 1 recap)
- Outer agent keeps ownership; delegates a subtask
- Control returns to the outer agent when the inner agent finishes

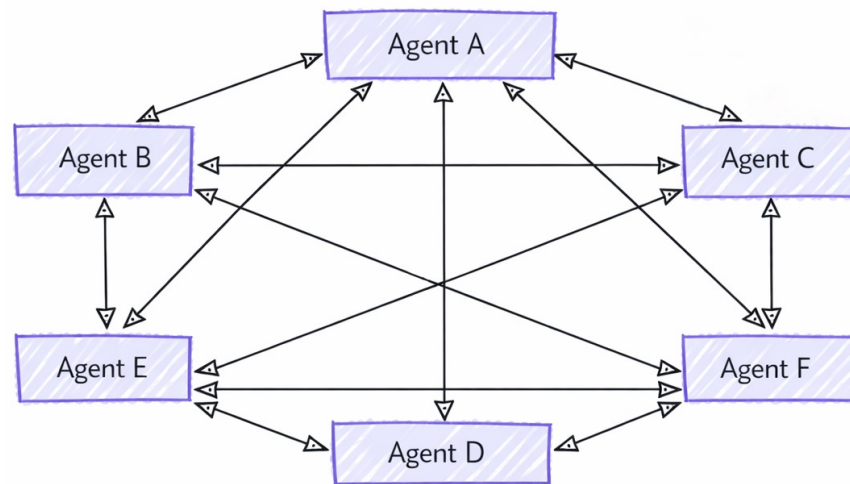
## Handoff

- The receiving agent takes full ownership of the task
- No central orchestrator — agents connect directly in a mesh topology
- Good for customer support, expert systems, dynamic routing



# Handoff — a mesh, not a hierarchy

- Agents connect directly to each other — a mesh, not a star with a middle orchestrator
- Any agent can hand the conversation fully to another agent it's configured to reach
- Full conversation history travels with the handoff — the receiving agent has complete context
- Interactive by default: control returns to the user whenever an agent responds without handing off



# Handoff in code

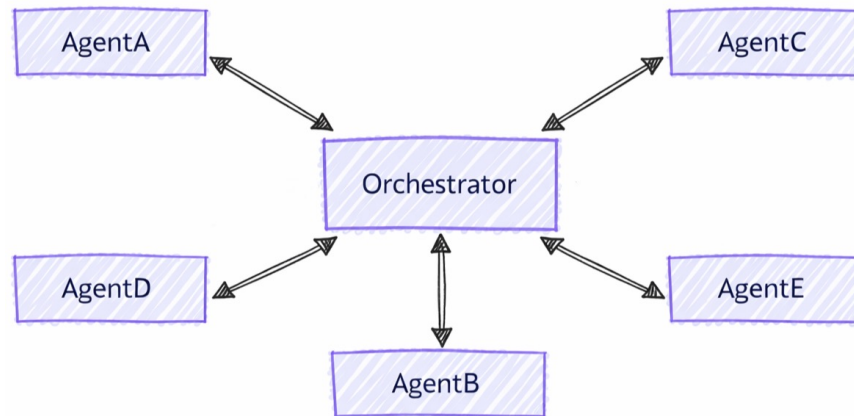
```
from agent_framework.orchestrations import HandoffBuilder

workflow = (
    HandoffBuilder(
        name="customer_support",
        participants=[triage_agent, refund_agent, order_agent, return_agent],
    )
    .with_start_agent(triage_agent)
    .add_handoff(triage_agent, [order_agent, return_agent])
    .add_handoff(return_agent, [refund_agent])
    .build()
)
```

Handoff is interactive by default — when an agent responds without handing off, control returns to the user for the next input.

# Group Chat — a coordinated conversation

- Star topology: an orchestrator in the middle selects who speaks next (round-robin, prompt-based, or custom logic)
- Agents can review and build on each other's responses across multiple rounds
- Context Synchronization: agents do NOT share one AgentSession instance — the orchestrator broadcasts each response so every agent's own session stays current before its next turn
- Bounded by a maximum round/iteration count



# When to use Group Chat

## Good fit

- Iterative refinement across multiple rounds
- Collaborative problem-solving, multi-perspective analysis
- Writer-reviewer content creation

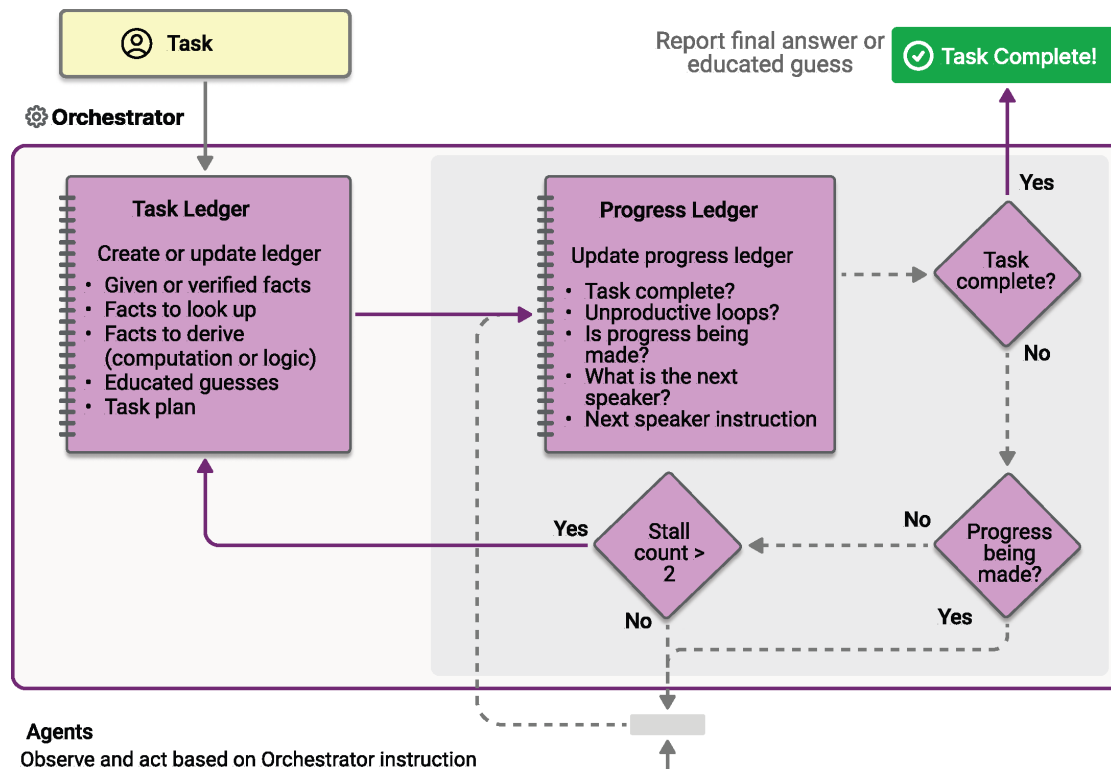
## Consider an alternative when

- You need strict sequential processing → Sequential
- Agents should work fully independently → Concurrent
- You need direct agent-to-agent handoffs → Handoff
- You need complex dynamic planning → Magentic

# Magentic — a planning manager

- Same star-topology architecture as Group Chat, with a more powerful, planning-based manager
- Based on AutoGen's Magentic-One design; well suited to open-ended tasks where the solution path isn't known in advance
- Framework's own caveat: `"it is untested how well the Magentic orchestration will perform outside of the original Magentic-One design"`
- If your scenario doesn't need complex planning, the docs themselves recommend Group Chat instead

# Magentic — task and progress ledgers



# Magentic's built-in guardrails

```
from agent_framework.orchestrations import MagenticBuilder

workflow = MagenticBuilder(
    participants=[researcher_agent, coder_agent],
    manager_agent=manager_agent,
    max_round_count=10,
    max_stall_count=3,
    max_reset_count=2,
).build()
```

A progress ledger tracks whether the team is making progress; consecutive non-progressing rounds increment a stall counter, and exceeding `max_stall_count` triggers an automatic reset and replan (up to `max_reset_count` times) — a concrete, built-in instance of the guardrail pattern Module 6 covers generically.

# Strengths, weaknesses, and cost profile

| Pattern    | Predictability                         | Typical cost driver                                                                           |
|------------|----------------------------------------|-----------------------------------------------------------------------------------------------|
| Sequential | High — fixed order every run           | Sum of every agent's tokens; grows if full context is chained                                 |
| Concurrent | High — all agents always run           | Sum of every agent's tokens, but latency is the slowest agent, not the sum                    |
| Handoff    | Low — path depends on the conversation | Scales with how many handoffs actually occur; hard to predict up front                        |
| Group Chat | Medium — bounded by max rounds         | Grows with round count; every round re-broadcasts full context to every agent                 |
| Magentic   | Lowest — planning can replan           | Highest and least predictable; planning/replanning adds overhead beyond the specialist agents |



# Takeaways

- ✓ You pick a pattern by asking who controls the next step and whether the path is fixed or dynamic.
- ✓ Sequential and Concurrent are predictable and cheap to reason about; Handoff, Group Chat, and Magentic trade predictability for flexibility.
- ✓ Sequential cannot loop back to an earlier agent — that gap is exactly what Module 3's custom workflow graphs are for.
- ✓ Magentic's `max_round_count`/`max_stall_count`/`max_reset_count` are a real, shipped example of the guardrails Module 6 covers next.